Day 42

Constructors in JavaScript:

What is a Constructor?

A constructor is a special function inside a class that automatically runs when you create an object. Its job is to initialize the object (give it default values or setup).

In very simple words:
A constructor prepares the object when it is created.

Why do we use Constructors?

- To set initial values (name, age, model, etc.)
- To make sure every object starts with correct properties
- To avoid writing setup code again and again

Constructor Syntax

```
class Person {
   constructor(name, age) {
      this.name = name;
      this.age = age;
   }
}
```

- constructor() → special method
- this.name → property of the object
- this.age → property of the object

Example: constructor only in employee class

```
JS main.js > Employee
1   class Employee {
2       constructor(name, age) {
3           this.name = name;
4           this.age = age;
5       }
6
7       getInfo() {
8           return `${this.name} is ${this.age} years old.`;
9       }
10  }
11  class Programmer extends Employee {
12      // No constructor → automatically uses parent's constructor
13  }
14  const p1 = new Programmer("Alice", 25);
15  console.log(p1.getInfo());

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day41-50\Day42> node main.js
Alice is 25 years old.
```

This code defines an Employee class with a constructor that sets a name and age, and a method getInfo() that returns a short description. The Programmer class extends Employee, but because it has no constructor of its own, it automatically uses the parent class's constructor. When you create new Programmer("Alice", 25), the values are passed to the Employee constructor, and getInfo() works because Programmer inherits all properties and methods from Employee.

Example: Constructor in BOTH Employee and Programmer

```
class Employee {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
class Programmer extends Employee {
    constructor(name, age, language) {
        super(name, age);  // must call super() first
        this.language = language;
    }
    getDetails() {
        return `${this.name} is a ${this.age}-year-old programmer who codes in ${this.language}.`;
    }
}
const p2 = new Programmer("Bob", 30, "JavaScript");
console.log(p2.getDetails());
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day41-50\Day42> node main.js
Bob is a 30-year-old programmer who codes in JavaScript.
```

This code defines an Employee class with basic properties, and a Programmer class that extends it by adding a new property, language. Since Programmer has its own constructor, it must call super(name, age) to run the Employee constructor before adding its own fields. When new Programmer("Bob", 30, "JavaScript") is created, the object gets all Employee properties plus the language field, and getDetails() returns a descriptive sentence using all three values.

Example: Child Constructor Overriding Parent Constructor

```
class Employee {
    constructor(name) {
        this.name = name;
    }
}
class Programmer extends Employee {
    constructor(name, level) {
        super(name); // still must be called
        this.level = level;
    }
    promote() {
        this.level++;
    }
}
const p3 = new Programmer("Charlie", 1);
console.log(p3.name);     // Charlie
console.log(p3.level);    // 1
p3.promote();
console.log(p3.level);    // 2
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day41-50\Day42> node main.js
Charlie
1
2
```

This code shows a parent Employee class with a single name property, and a child Programmer class that extends it by adding a level property. The Programmer constructor must call super(name) to initialize the inherited name field before setting level. The class also includes a promote() method that increases the programmer's level. When a new Programmer object is created with "Charlie" and level 1, it has both the inherited name and its own level, and calling promote() updates that level to 2.

Example: Constructor with default values

```
55  class Employee {
56      constructor(name = "Unknown", salary = 0) {
57          this.name = name;
58          this.salary = salary;
59      }
60  }
61  class Programmer extends Employee {
62      constructor(name, salary, language = "JavaScript") {
63          super(name, salary);
64          this.language = language;
65      }
66  }
67  const p4 = new Programmer("Derrick", 50000);
68  console.log(p4);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day41-50\Day42> node main.js
Programmer { name: 'Derrick', salary: 50000, language: 'JavaScript' }
```

This code uses default values in constructors. The Employee class assigns default values for name and salary if none are provided. The Programmer class extends Employee and adds a new property, language, which also has a default value of "JavaScript". When creating new Programmer("Derrick", 50000), only name and salary are passed, so the inherited fields are set accordingly, and language automatically becomes "JavaScript". The resulting object contains all three properties: name, salary, and language.

Short Summary

- A constructor is a special method used to set up an object.
- It runs automatically when new is used.
- It initializes properties using this.
- super() is needed when using constructors in inheritance.
- If you don’t write a constructor, JavaScript creates one for you.

--The End--